

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	Total Marks:	100
Student Name:	Rajesh Kumar V	Reg. No.:	192511289
Section / Batch:	_____	Date:	_____

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

(To be completed and signed personally, based on work you understand and can explain.)

Experiment 1: Merge Sort

Aim: To design and implement Merge Sort using the Divide and Conquer approach.

Algorithm:

- Divide: Split the array into two halves at the midpoint.
- Conquer: Recursively sort each half.
- Combine: Merge the two sorted halves into a single sorted array.

```
MERGE_SORT(A, left, right):
    if left < right:
        mid = (left + right) / 2
        MERGE_SORT(A, left, mid)
        MERGE_SORT(A, mid+1, right)
        MERGE(A, left, mid, right)

MERGE(A, left, mid, right):
    create temp arrays L = A[left..mid], R = A[mid+1..right]
    i = j = 0; k = left
    while both L and R have elements:
        place the smaller of L[i], R[j] into A[k]; advance pointers
    copy remaining elements of L (if any) into A
    copy remaining elements of R (if any) into A
```

Python Implementation:

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

if __name__ == "__main__":
    data = [38, 27, 43, 3, 9, 82, 10]
    print("Input :", data)
    print("Sorted:", merge_sort(data))
```

Sample Output:

```
Input : [38, 27, 43, 3, 9, 82, 10]
Sorted: [3, 9, 10, 27, 38, 43, 82]
```

Complexity Analysis:

- Recurrence: $T(n) = 2T(n/2) + O(n) \rightarrow$ by the Master Theorem, $T(n) = O(n \log n)$.
- Time Complexity: Best = Average = Worst = $O(n \log n)$.
- Space Complexity: $O(n)$ for the temporary arrays used during merging.

- Stability: Merge Sort is a stable sorting algorithm.

Experiment 2: Quick Sort

Aim: To design and implement Quick Sort using the Divide and Conquer approach.

Algorithm:

- Divide: Choose a pivot and partition the array so smaller elements go left, larger go right.
- Conquer: Recursively sort the left and right partitions.
- Combine: No explicit merge step needed — the array is sorted in place once partitions are sorted.

```
QUICK_SORT(A, low, high):
    if low < high:
        p = PARTITION(A, low, high)
        QUICK_SORT(A, low, p-1)
        QUICK_SORT(A, p+1, high)

PARTITION(A, low, high):
    pivot = A[high]
    i = low - 1
    for j = low to high-1:
        if A[j] <= pivot:
            i = i + 1; swap A[i], A[j]
    swap A[i+1], A[high]
    return i + 1
```

Python Implementation:

```
def quick_sort(arr, low=0, high=None):
    if high is None:
        high = len(arr) - 1
    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)
    return arr

def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

if __name__ == "__main__":
    data = [10, 7, 8, 9, 1, 5]
    print("Input :", data)
    print("Sorted:", quick_sort(data))
```

Sample Output:

```
Input : [10, 7, 8, 9, 1, 5]
Sorted: [1, 5, 7, 8, 9, 10]
```

Complexity Analysis:

- Best/Average case: $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$, when the pivot splits the array evenly.
- Worst case: $T(n) = T(n-1) + O(n) \rightarrow O(n^2)$, when the pivot is always the smallest/largest element (e.g. already-sorted input with last-element pivot).

- Space Complexity: $O(\log n)$ average (recursion stack), $O(n)$ worst case.
- Quick Sort is not stable, but is generally faster in practice than Merge Sort due to in-place partitioning and better cache performance.

Experiment 3: Binary Search

Aim: To design and implement Binary Search using the Divide and Conquer approach.

Algorithm:

- Divide: Compare the target with the middle element of the sorted array.
- Conquer: Recurse into the left half if the target is smaller, or the right half if larger.
- Combine: Trivial — the search terminates when the element is found or the range is empty.

```
BINARY_SEARCH(A, low, high, key):  
    if low > high:  
        return -1  
    mid = (low + high) / 2  
    if A[mid] == key:  
        return mid  
    else if A[mid] > key:  
        return BINARY_SEARCH(A, low, mid-1, key)  
    else:  
        return BINARY_SEARCH(A, mid+1, high, key)
```

Python Implementation:

```
def binary_search(arr, key, low=0, high=None):  
    if high is None:  
        high = len(arr) - 1  
    if low > high:  
        return -1  
    mid = (low + high) // 2  
    if arr[mid] == key:  
        return mid  
    elif arr[mid] > key:  
        return binary_search(arr, key, low, mid - 1)  
    else:  
        return binary_search(arr, key, mid + 1, high)  
  
if __name__ == "__main__":  
    data = [2, 5, 8, 12, 16, 23, 38, 45, 56, 72]  
    key = 23  
    idx = binary_search(data, key)  
    print(f"Element {key} found at index {idx}" if idx != -1 else "Not found")
```

Sample Output:

```
Element 23 found at index 5
```

Complexity Analysis:

- Recurrence: $T(n) = T(n/2) + O(1) \rightarrow$ by the Master Theorem, $T(n) = O(\log n)$.
- Time Complexity: $O(\log n)$ for best, average, and worst case.
- Space Complexity: $O(\log n)$ for recursive version (call stack); $O(1)$ for an iterative version.
- Precondition: the input array must be sorted.

Experiment 4: Matrix Multiplication (Strassen's Algorithm)

Aim: To design and implement Divide and Conquer based Matrix Multiplication (Strassen's Algorithm) and compare it with the standard method.

Standard Divide and Conquer Method:

Split each $n \times n$ matrix into four $(n/2) \times (n/2)$ submatrices and combine 8 recursive multiplications:

```
C11 = A11*B11 + A12*B21    C12 = A11*B12 + A12*B22
C21 = A21*B11 + A22*B21    C22 = A21*B12 + A22*B22
Recurrence:  $T(n) = 8T(n/2) + O(n^2) \rightarrow O(n^3)$  (no better than standard method)
```

Strassen's Algorithm (reduces 8 multiplications to 7):

```
M1 = (A11 + A22) * (B11 + B22)
M2 = (A21 + A22) * B11
M3 = A11 * (B12 - B22)
M4 = A22 * (B21 - B11)
M5 = (A11 + A12) * B22
M6 = (A21 - A11) * (B11 + B12)
M7 = (A12 - A22) * (B21 + B22)

C11 = M1 + M4 - M5 + M7
C12 = M3 + M5
C21 = M2 + M4
C22 = M1 - M2 + M3 + M6
```

Python Implementation:

```
import numpy as np

def split(M):
    n = len(M) // 2
    return (M[:n, :n], M[:n, n:], M[n:, :n], M[n:, n:])

def strassen(A, B):
    n = A.shape[0]
    if n == 1:
        return A * B
    A11, A12, A21, A22 = split(A)
    B11, B12, B21, B22 = split(B)

    M1 = strassen(A11 + A22, B11 + B22)
    M2 = strassen(A21 + A22, B11)
    M3 = strassen(A11, B12 - B22)
    M4 = strassen(A22, B21 - B11)
    M5 = strassen(A11 + A12, B22)
    M6 = strassen(A21 - A11, B11 + B12)
    M7 = strassen(A12 - A22, B21 + B22)

    C11 = M1 + M4 - M5 + M7
    C12 = M3 + M5
    C21 = M2 + M4
    C22 = M1 - M2 + M3 + M6

    top = np.hstack((C11, C12))
    bottom = np.hstack((C21, C22))
    return np.vstack((top, bottom))

if __name__ == "__main__":
```

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print("Result:\n", strassen(A, B))
```

Sample Output:

```
Result:
[[19 22]
 [43 50]]
```

Complexity Analysis:

- Standard (naive) method: $T(n) = O(n^3)$.
- Standard Divide and Conquer (8 sub-multiplications): $T(n) = 8T(n/2) + O(n^2) = O(n^3)$.
- Strassen's Algorithm: $T(n) = 7T(n/2) + O(n^2) = O(n^{\log_2(7)}) \approx O(n^{2.81})$, an asymptotic improvement over $O(n^3)$.
- Trade-off: Strassen's has higher constant factors and lower numerical stability, so it typically outperforms the naive method only for large n .

Conclusion

All four experiments were implemented using the Divide and Conquer paradigm: reduce a problem into smaller subproblems of the same type, solve them recursively, and combine the results. Merge Sort and Binary Search achieve their efficiency through balanced splitting, Quick Sort through in-place partitioning around a pivot, and Strassen's algorithm through an algebraic identity that trades extra additions for fewer recursive multiplications.